

Facial Emotion Recognition Using CNN

1. Project Overview

Facial Emotion Recognition is a Computer Vision and Deep Learning project that aims to automatically identify human emotions from facial images.

In this project, a **Convolutional Neural Network (CNN)** is developed using TensorFlow/Keras to classify facial expressions into **seven different emotion categories**:

- Angry
- Disgust
- Fear
- Happy
- Neutral
- Sad
- Surprise

The model takes a facial image as input, processes the image through multiple convolutional and pooling layers, and finally predicts the most likely emotion using a **7-class Softmax output layer**.

2. Dataset

The dataset used in this project is the **Facial Emotion Recognition Dataset** available on Kaggle.

Dataset Source

Kaggle Dataset:

<https://www.kaggle.com/datasets/fahadullaha/facial-emotion-recognition-dataset>

The dataset was provided as a data source for a university project and was processed using the **FER2013** and **RAFDB** datasets.

According to the dataset description, several techniques were used during dataset preparation, including **HaarCascade face filtering with a 0.8 confidence score** to filter images containing faces and reduce noise.

The grayscale images from FER2013 were also converted to RGB.

The resulting dataset contains facial images representing **7 different emotions**.

Dataset License

The dataset is provided under:

Creative Commons Attribution 4.0 International (CC BY 4.0)

License:

<https://creativecommons.org/licenses/by/4.0/>

3. Dataset Structure

The dataset contains a directory called `processed_data`.

The directory contains seven emotion folders:

```
processed_data/
|
|— angry/
|— disgust/
|— fear/
|— happy/
|— neutral/
|— sad/
|— surprise/
```

Each folder contains images belonging to its corresponding emotion.

The folder name is used as the class label.

Dataset Classes

Class	Emotion
0	Angry
1	Disgust
2	Fear
3	Happy
4	Neutral

5 Sad

6 Surprise

The actual class names detected by the notebook are:

['angry', 'disgust', 'fear', 'happy',
'neutral', 'sad', 'surprise']

4. Dataset Size

The notebook detected:

49,779 total images

The dataset was automatically divided into:

- **39,824 images — Training set**
- **9,955 images — Validation set**

The split used was:

80% Training / 20% Validation

The notebook used:

`validation_split=0.2`

Therefore:

Total Images = 49,779
Training Images = 39,824
Validation Images = 9,955

No separate test dataset was created in the notebook.

5. Dataset Class Distribution

According to the Kaggle dataset information, the processed dataset contains approximately:

Emotion Number of Images

Angry	5,920
Disgust	5,920
Fear	5,920
Happy	11,400
Neutral	8,166
Sad	6,535
Surprise	5,920

The dataset description specifically warns that directly using the dataset can result in problems because of **class imbalance**.

For example, the **happy** class contains substantially more images than some of the other classes.

This imbalance can affect model performance because the model may learn some classes better than others.

6. Dataset Path

The dataset was loaded from the following Kaggle path:

```
dataset_path = "/kaggle/input/datasets/fahadullaha/facial-emotion-recognition-dataset/processed_data"
```

The notebook first checks the available emotion directories using:

```
print(os.listdir(dataset_path))
```

7. Technologies and Libraries

The project uses Python and TensorFlow/Keras for deep learning.

Libraries Used

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
import os
```

Main Libraries

Library	Purpose
TensorFlow	Deep Learning framework
Keras	Building and training the CNN
Matplotlib	Visualization
NumPy	Numerical operations
OS	Dataset/file handling

8. Image Loading

The images are loaded using TensorFlow's:

```
tf.keras.utils.image_dataset_from_directory()
```

This function automatically:

1. Reads images from the emotion folders.
2. Uses folder names as labels.
3. Resizes images.
4. Creates batches.
5. Splits the dataset into training and validation sets.

The notebook uses:

```
IMG_SIZE = (48, 48)
BATCH_SIZE = 32
```

Therefore, every image is resized to:

48 × 48 pixels

with:

3 color channels (RGB)

The CNN input shape is therefore:

$48 \times 48 \times 3$

9. Training and Validation Split

The dataset is divided using:

`validation_split=0.2`

For training:

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
```

For validation:

```
val_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
```

Why `seed=42`?

The seed ensures that the dataset splitting process is reproducible.

10. Image Visualization

Before training the model, sample images from the training dataset are visualized.

The notebook displays **9 sample images** in a 3×3 grid.

```
plt.figure(figsize=(10, 10))

for images, labels in train_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
```

This step helps verify that:

- Images are loading correctly.
 - Images belong to the expected classes.
 - Labels are correctly assigned.
 - Images have the expected visual appearance.
-

11. Image Normalization

The images initially contain pixel values approximately between:

0 – 255

The notebook applies normalization using:

```
normalization_layer = layers.Rescaling(1./255)
```

This converts pixel values approximately to:

0 – 1

For example:

255 → 1.0

128 → 0.502

0 → 0.0

Normalization makes the input values smaller and generally helps neural networks train more effectively.

The normalization is applied to both training and validation datasets:

```
train_ds = train_ds.map(
    lambda x, y: (normalization_layer(x), y)
)
```

```
val_ds = val_ds.map(
```

```
lambda x, y: (normalization_layer(x), y)  
)
```

12. CNN Architecture

The project uses a custom **Convolutional Neural Network (CNN)**.

The architecture consists of:

```
Input Image  
↓  
Conv2D 32 filters  
↓  
MaxPooling  
↓  
Conv2D 64 filters  
↓  
MaxPooling  
↓  
Conv2D 128 filters  
↓  
MaxPooling  
↓  
Flatten  
↓  
Dense 128  
↓  
Dropout 0.5  
↓  
Dense 7 + Softmax  
↓  
Emotion Prediction
```

13. CNN Model Code

The model implemented in the notebook is:

```
model = models.Sequential([  
  
    layers.Conv2D(  
        32,  
        (3, 3),  
        activation='relu',
```

```
        input_shape=(48, 48, 3)
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64,
        (3, 3),
        activation='relu'
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        128,
        (3, 3),
        activation='relu'
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(
        128,
        activation='relu'
    ),

    layers.Dropout(0.5),

    layers.Dense(
        7,
        activation='softmax'
    )
])
```

14. Layer-by-Layer Explanation

14.1 Input Layer

The input image size is:

$48 \times 48 \times 3$

Where:

- 48 = image height
 - 48 = image width
 - 3 = RGB channels
-

14.2 First Convolutional Layer

```
layers.Conv2D(  
    32,  
    (3, 3),  
    activation='relu'  
)
```

This layer contains:

32 filters
3 × 3 kernel
ReLU activation

The purpose of convolution is to detect important visual patterns such as:

- Edges
- Lines
- Corners
- Simple facial features

The output shape becomes:

46 × 46 × 32

15. First MaxPooling Layer

```
layers.MaxPooling2D((2, 2))
```

MaxPooling reduces the spatial dimensions.

The output changes from:

46 × 46 × 32

to:

23 × 23 × 32

Pooling helps reduce computational complexity while retaining important features.

16. Second Convolutional Layer

```
layers.Conv2D(  
    64,  
    (3, 3),  
    activation='relu'  
)
```

This layer uses:

64 filters
3 × 3 kernel
ReLU activation

The output shape becomes:

21 × 21 × 64

As the network becomes deeper, it can learn more complex facial patterns.

17. Second MaxPooling Layer

```
layers.MaxPooling2D((2, 2))
```

The output changes from:

21 × 21 × 64

to:

10 × 10 × 64

18. Third Convolutional Layer

```
layers.Conv2D(  
    128,  
    (3, 3),  
    activation='relu'  
)
```

This layer contains:

128 filters
3 × 3 kernel

ReLU activation

Output:

$8 \times 8 \times 128$

This deeper layer can learn more complex features related to facial expressions.

19. Third MaxPooling Layer

The third pooling layer:

`layers.MaxPooling2D((2, 2))`

reduces:

$8 \times 8 \times 128$

to:

$4 \times 4 \times 128$

20. Flatten Layer

The Flatten layer converts the 3D feature maps into a 1D vector.

`layers.Flatten()`

The feature representation becomes:

$4 \times 4 \times 128 = 2048$

Therefore, the Flatten layer produces:

2048 features

21. Dense Layer

The model then uses:

`layers.Dense(
128,`

```
    activation='relu'  
)
```

This fully connected layer contains:

128 neurons

It uses ReLU activation to learn higher-level combinations of the extracted facial features.

22. Dropout Layer

The model uses:

```
layers.Dropout(0.5)
```

This means that during training, approximately **50% of the neurons** in this layer are randomly dropped.

The purpose is to reduce:

Overfitting

and encourage the model to learn more robust features.

23. Output Layer

The final layer is:

```
layers.Dense(  
    7,  
    activation='softmax'  
)
```

There are 7 neurons because the dataset contains seven emotion classes.

The Softmax function converts the model's outputs into probabilities.

For example, a prediction might look conceptually like:

```
Angry    → 0.05  
Disgust  → 0.02  
Fear     → 0.10  
Happy    → 0.70
```

Neutral → 0.08
Sad → 0.03
Surprise → 0.02

The highest probability would be:

Happy

Therefore, the model predicts **Happy**.

24. Model Parameters

According to the notebook's model summary:

Layer	Parameters
Conv2D — 32 filters	896
Conv2D — 64 filters	18,496
Conv2D — 128 filters	73,856
Dense — 128 neurons	262,272
Output Dense — 7 neurons	903
Total	356,423

The model contains:

Total Parameters: 356,423
Trainable Parameters: 356,423
Non-trainable Parameters: 0

The model size shown by TensorFlow is approximately:

1.36 MB

25. Model Compilation

The model is compiled using:

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

Three important components are used.

Optimizer

Adam

Adam is used to update the model's weights during training.

Loss Function

Sparse Categorical Crossentropy

This is suitable because the problem is a multi-class classification problem with integer class labels.

Evaluation Metric

Accuracy

Accuracy measures the percentage of correctly classified images.

26. Model Training

The model is trained using:

```
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=20  
)
```

Training configuration:

Parameter	Value
Training images	39,824

Validation images	9,955
Batch size	32
Epochs	20
Optimizer	Adam
Loss	Sparse Categorical Crossentropy
Metric	Accuracy
Image size	48 × 48 × 3

27. Training Results

The model was trained for 20 epochs.

Final Training Results

At Epoch 20:

Training Accuracy: 71.74%

Training Loss: 0.7233

Validation Accuracy: 60.67%

Validation Loss: 1.1746

The best validation accuracy recorded during training was approximately:

61.92%

which occurred at:

Epoch 14

At Epoch 14:

Training Accuracy: 67.78%

Validation Accuracy: 61.92%

28. Training Performance

The training accuracy improved throughout the 20 epochs:

Epoch 1 → 32.23%
Epoch 5 → 57.93%
Epoch 10 → 64.55%
Epoch 15 → 68.64%
Epoch 20 → 71.74%

This shows that the CNN successfully learned facial features from the training data.

However, validation accuracy did not continuously increase.

It reached approximately:

61.92%

at Epoch 14 and then fluctuated.

29. Training vs Validation Accuracy

The notebook generates an accuracy graph using:

```
plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)
```

The graph compares:

- Training Accuracy
- Validation Accuracy

This is useful for understanding model learning and detecting possible overfitting.

30. Interpretation of Results

The final training accuracy was:

71.74%

while the final validation accuracy was:

60.67%

There is a noticeable difference between training and validation performance.

This suggests that the model is learning the training data better than it generalizes to unseen validation images.

The increasing training accuracy combined with fluctuating validation accuracy indicates that **some degree of overfitting may be occurring**.

The dataset's documented class imbalance may also contribute to differences in performance among emotion classes.

31. Important Limitation of the Current Notebook

Although the dataset documentation mentions class imbalance, the current notebook does **not** implement a class-balancing technique.

The notebook does not currently use:

- Class weights
- Oversampling
- Undersampling
- SMOTE
- Balanced batch sampling

Therefore, the model is trained directly on the available class distribution.

For a stronger version of the project, class imbalance should be addressed.

32. Data Augmentation

The current notebook does **not** use image augmentation.

For example, the current implementation does not include:

RandomFlip
RandomRotation
RandomZoom
RandomContrast

Adding augmentation could help the model generalize better to different facial orientations and appearances.

33. Test Dataset

The notebook creates:

Training Dataset
Validation Dataset

It does not create a separate:

Test Dataset

Therefore, the reported validation accuracy should not be described as the final independent test accuracy.

A separate test set would provide a better estimate of the model's performance on completely unseen data.

34. Confusion Matrix

The current notebook does not calculate a confusion matrix.

A confusion matrix would be useful for determining which emotions the model confuses with each other.

For example, it could show whether:

Fear → Sad
Sad → Neutral
Angry → Disgust

are frequently confused.

This would provide more detailed information than overall accuracy alone.

35. Classification Report

The current notebook also does not calculate:

- Precision
- Recall
- F1-score

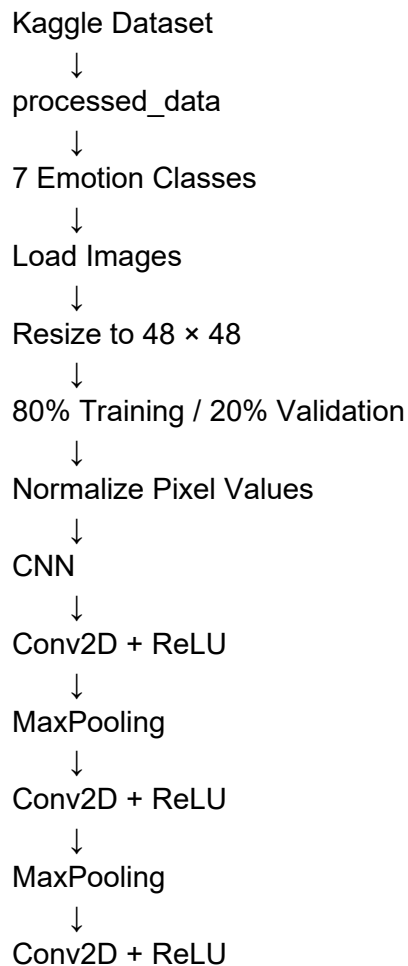
for individual emotion classes.

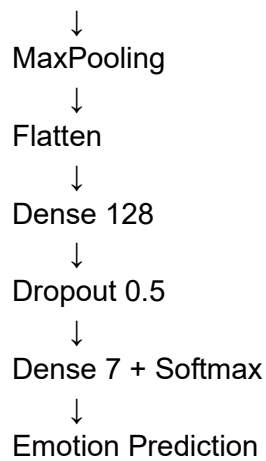
These metrics would be useful because the dataset contains class imbalance.

Overall accuracy alone may not accurately represent how well the model performs on each individual emotion.

36. Complete Project Workflow

The overall workflow of the project is:





37. Input and Output

Input

The model accepts:

48 × 48 RGB facial image

Output

The model produces probabilities for seven emotions:

Angry
Disgust
Fear
Happy
Neutral
Sad
Surprise

The class with the highest probability becomes the predicted emotion.

38. Applications

Facial emotion recognition can be used in several areas:

Human-Computer Interaction

Systems can respond differently depending on the user's facial expression.

Education

Emotion recognition can potentially be used to study learner engagement and reactions.

Customer Experience

Businesses can analyze facial reactions during user studies.

Entertainment

Applications can adapt content according to detected facial expressions.

Social Robots

Robots can use facial emotion information to improve human interaction.

Computer Vision Research

The project provides practical experience with image classification and CNNs.

39. Strengths of the Project

The project has several strengths:

- Uses a relatively large facial emotion dataset.
 - Contains seven emotion classes.
 - Uses a CNN specifically designed for image classification.
 - Uses RGB images.
 - Applies image normalization.
 - Uses validation data during training.
 - Uses Dropout to help reduce overfitting.
 - Uses Adam optimization.
 - Provides training and validation accuracy visualization.
 - Contains 356,423 trainable parameters.
-

40. Limitations

The current implementation has some limitations:

1. No separate test dataset.
2. No class balancing.

3. No data augmentation.
 4. No confusion matrix.
 5. No classification report.
 6. No precision, recall, or F1-score.
 7. Validation accuracy remains around 60–62%.
 8. The model may overfit the training data.
 9. Only one CNN architecture is evaluated.
 10. No transfer learning model is tested.
-

41. Future Improvements

The project can be improved by implementing:

1. Data Augmentation

Add:

Random Flip
Random Rotation
Random Zoom
Random Contrast

to increase training diversity.

2. Class Weighting

Class weights can be used to give more importance to underrepresented emotion classes.

3. Separate Test Dataset

A completely unseen test dataset should be used for final evaluation.

4. Confusion Matrix

A confusion matrix should be generated to identify incorrectly classified emotions.

5. Classification Report

Precision, recall, and F1-score should be calculated for every emotion.

6. Early Stopping

Early stopping can prevent unnecessary training once validation performance stops improving.

7. Learning Rate Scheduling

A learning-rate scheduler can help the model converge more effectively.

8. Batch Normalization

Batch normalization can potentially improve training stability.

9. Transfer Learning

Pretrained CNN architectures such as MobileNet, EfficientNet, or ResNet can be tested.

10. Hyperparameter Tuning

Different values for:

- Learning rate
- Batch size
- Number of filters
- Dropout
- Number of layers
- Image resolution

can be tested.

42. Conclusion

This project successfully implements a **Convolutional Neural Network for facial emotion recognition** using the Facial Emotion Recognition Dataset.

The dataset contains **49,779 images across seven emotion classes**, and the notebook divides these images into **39,824 training images and 9,955 validation images**.

The CNN consists of three convolutional blocks followed by a Flatten layer, a Dense layer, Dropout, and a seven-neuron Softmax output layer.

After 20 epochs of training, the model achieved:

Training Accuracy: 71.74%

Validation Accuracy: 60.67%

The highest validation accuracy observed during training was approximately:

61.92%

The results demonstrate that the CNN is able to learn meaningful facial-expression features. However, the difference between training and validation performance suggests that the model could benefit from additional techniques such as data augmentation, class balancing, regularization, and a more advanced architecture.

Overall, this project provides a practical implementation of **Deep Learning, Computer Vision, Image Classification, and CNN-based Facial Emotion Recognition**.

43. Project Information

Project: Facial Emotion Recognition using CNN

Task: Multi-Class Image Classification

Domain: Computer Vision / Deep Learning

Framework: TensorFlow / Keras

Dataset: Facial Emotion Recognition Dataset

Dataset Source: Kaggle

Classes: 7

Total Images: 49,779

Training Images: 39,824

Validation Images: 9,955

Image Size: $48 \times 48 \times 3$

Batch Size: 32

Epochs: 20

Optimizer: Adam

Loss Function: Sparse Categorical Crossentropy

Final Training Accuracy: 71.74%

Final Validation Accuracy: 60.67%

Best Validation Accuracy: 61.92%

Model Parameters: 356,423

44. Dataset Reference

The dataset used in this project is available on Kaggle:

Facial Emotion Recognition Dataset — Fahadullaha

<https://www.kaggle.com/datasets/fahadullaha/facial-emotion-recognition-dataset>

The dataset was processed using data originating from **FER2013** and **RAFDB**, with facial filtering and preprocessing techniques described by the dataset author.

The dataset is licensed under **CC BY 4.0**.

License:

<https://creativecommons.org/licenses/by/4.0/>

45. Repository Description

Short Description

A CNN-based Facial Emotion Recognition project using TensorFlow/Keras to classify facial expressions into seven emotion categories: Angry, Disgust, Fear, Happy, Neutral, Sad, and Surprise.

Technologies

Python
TensorFlow
Keras
CNN
Computer Vision
NumPy
Matplotlib
Kaggle
Deep Learning
Image Classification

Emotion Classes

Angry
Disgust
Fear

Happy
Neutral
Sad
Surprise

Model Architecture

Input (48×48×3)



Conv2D (32)



MaxPooling



Conv2D (64)



MaxPooling



Conv2D (128)



MaxPooling



Flatten



Dense (128)



Dropout (0.5)



Dense (7, Softmax)